

Lecture 10 Part 1

Knuth-Morris-Pratt Algorithm

CSEN 1038 Advanced Data Structures and Algorithms

Table of contents

1. Rabin-Karp Revisited
2. Prefix Function
3. Prefix Function Usage
4. KMP Algorithm
5. Applications
6. (Optional) KMP Automaton

Rabin-Karp Revisited

Our Problem

Objective: Locate all occurrences of a specific pattern string within a larger text string efficiently.

Formal Definition:

- Let T be a **text** string of length n
- Let P be a **pattern** string of length m
- Assume $m \leq n$.

Goal: Find all valid *shifts* i ($0 \leq i \leq n - m$) such that the substring of T starting at i matches P exactly:

$$T[i \dots i + m - 1] = P$$

The Rabin Karp Algorithm

- Transforms the string matching problem into a **numeric equality** problem.
- Utilizes *Rolling Hashing* to compute the hash value of P and each m -length substring of T in $\mathcal{O}(1)$ rolling time.
- Expected Time Complexity: $\mathcal{O}(n + m)$

Prefix Function

Prefix Function (π)

Definition:

For a given string s of length n , the prefix function $\pi[i]$ is defined as the length of the **longest proper suffix** of the substring $s[0 \dots i]$ that is also a **prefix** of $s[0 \dots i]$.

Note: A “proper suffix” must be strictly shorter than the substring itself. By definition, $\pi[0] = 0$.

Example:

For the string $s = \text{“abababca”}$:

i	0	1	2	3	4	5	6	7
$s[i]$	a	b	a	b	a	b	c	a
$\pi[i]$	0	0	1	2	3	4	0	1

Prefix Function Usage

What if we can compute the prefix function efficiently?

The Concatenation Trick:

We can solve the pattern matching problem by reducing it to a single prefix function computation.

Algorithm Steps:

1. Construct a combined string S :

$$S = P + \# + T$$

Where P is the pattern, T is the text, and $\#$ is a unique separator.

2. Compute the prefix function π for the entire string S .
3. A match is found at index k of S if:

$$\pi[k] = |P|$$

Why the separator (#) is crucial: It ensures the prefix function value never exceeds the length of the pattern. This prevents a "suffix-prefix" match from spanning across both the pattern and the text incorrectly.

KMP Algorithm

Computing the Prefix Function: The Strategy

To compute $\pi[i]$ efficiently, we use the values already computed for $\pi[0 \dots i - 1]$.

Algorithm Steps:

- **Initialize:** Set $\pi[0] = 0$. We maintain a variable j representing the length of the current matching prefix.
- **Iterate:** For each index i from 1 to $n - 1$:
 1. **Try to extend:** Start with $j = \pi[i - 1]$. If $s[i] = s[j]$, we simply extend the match: $j = j + 1$.
 2. **Fallback on mismatch:** If $s[i] \neq s[j]$, we look for a smaller prefix. We “jump back” to the next best candidate: $j = \pi[j - 1]$. Repeat this until $s[i] = s[j]$ or $j = 0$.
 3. **Update:** Store the final value of j into $\pi[i]$.

Key Insight: We never re-calculate from scratch. We reuse the “failures” of previous prefixes to find the next one.

Prefix Function: Pseudocode

The following $O(n)$ implementation is concise and avoids redundant comparisons.

```
vector<int> compute_prefix(string s) {  
    int n = s.length();  
    vector<int> pi(n);  
    for (int i = 1; i < n; i++) {  
        int j = pi[i-1];  
        while (j > 0 && s[i] != s[j])  
            j = pi[j-1]; // Jump back  
        if (s[i] == s[j])  
            j++;  
        pi[i] = j;  
    }  
    return pi;  
}
```

Time Complexity: Amortized Analysis

At first glance, the nested while loop suggests $O(n^2)$. However, the actual complexity is **linear** $O(n)$.

Proof via Aggregate Analysis:

- Let j be our “potential” or a counter.
- **Increments:** In the for loop, the value of j can increase by **at most 1** per iteration (when $s[i] = s[j]$). Total increases $\leq n$.
- **Decrements:** The statement $j = \pi[j - 1]$ inside the while loop *always* strictly decreases j (since $\pi[k] < k + 1$).
- **Conclusion:** Since j starts at 0 and never goes below 0, the total number of times j can decrease cannot exceed the total number of times it has increased.

⇒ The while loop body executes at most n times across the **entire** algorithm.

⇒ Total Time Complexity: **$O(n)$** .

Applications

Finding the Smallest Period of a String

- **Goal:** Find the shortest substring that can be perfectly repeated to form the entire string s (length n).
- **Solution:** The prefix function gives us the longest matching prefix and suffix. The length of the smallest potential period is naturally $k = n - \pi[n - 1]$. If $n \bmod k = 0$, then k is the valid period length!

Frequency of Each Prefix as a Substring

- **Goal:** Count how many times each prefix of s appears anywhere within s .
- **Solution:** We can compute this in $O(n)$ by iterating backward through the π array. Whenever a prefix of length i occurs, its shorter “fallback” prefix of length $\pi[i - 1]$ also implicitly occurs. We simply accumulate these counts from right to left.

(Optional) KMP Automaton

The KMP Automaton

What is it?

The KMP Automaton is a pre-computed roadmap for the KMP algorithm. Instead of figuring out where to “jump back” on the fly during a mismatch, it calculates all possible moves in advance.

How it works:

- It builds a transition table telling you exactly which state to go to next for **every** possible character.
- This eliminates all backward jumping, ensuring every character is processed in exactly $O(1)$ time.

Questions?

- CP Algorithm:
<https://cp-algorithms.com/string/prefix-function.html>
- GUC Community Session:
<https://youtu.be/hxcbkuqQeZQ?si=P1a90Pkr4fNJnE6L>